TIP One of my first introductions to this idea was a series of posts by Jimmy Bogard. The following link points to the last post in the series, but it contains links to all the earlier posts too. Bogard is also behind the MediatR library (https://github.com/jbogard/MediatR), which makes creating thin controllers even easier. See “Put your controllers on a diet: POSTs and commands”: http://mng.bz/7VNQ.

All that said, controllers and actions are classes and methods, so you can write unit tests for them. The difficulty is deciding what you want to test. As an example, we’ll consider the simple API controller in the following listing, which converts a value using a provided exchange rate and returns a response.

Listing 36.4 The API controller under test

```
[Route("api/[controller]")]
public class CurrencyController : ControllerBase
{
    private readonly CurrencyConverter _converter          ❶
       = new CurrencyConverter();                          ❶

    [HttpGet]
    public ActionResult<decimal> Convert(InputModel model) ❷
    {
        if (!ModelState.IsValid)                           ❸
        {                                                  ❸
            return BadRequest(ModelState);                 ❸
        }                                                  ❸

        decimal result = _converter.ConvertToGbp(model)    ❹

        return result;                                     ❺
    }
}
```

❶ The CurrencyConverter would normally be injected using DI and is created here for simplicity.

❷ The Convert method returns an Action-Result<T>.

❸ If the input is invalid, returns a 400 Bad Request result, including the ModelState